

TDEW Estimation: PRAM Pseudocode and Complexity

Bucket-year anomaly local linear regression pipeline

Piero Palacios

2026-05-07

Table of contents

1 Purpose	2
2 PRAM Model	2
3 Symbols	2
4 How to Read the Complexity Formulas	3
5 Algorithm 1: Full TDEW Pipeline	4
6 Algorithm 2: Build Climatology	5
7 Algorithm 3: Build Bucket-Year Training Dataset	5
8 Algorithm 4: Shard Climatology by Bucket	7
9 Algorithm 4b: Shard Future TMIN by Bucket	7
10 Algorithm 5: Train Anomaly Coefficients	8
11 Algorithm 6: Check Completeness	11
12 Algorithm 7: Repair Failed DOYs	11
13 Algorithm 8: Patch Coefficients	12
14 Algorithm 9: Optional Forecasting (bucketed)	13
15 Old Design vs Bucket-Year Design	14
16 Summary Complexity Table	14
17 Overall Complexity	15
18 HPC Interpretation	16
19 Practical Notes for This Machine	17
19.1 Hardware is just more processors	17
20 Experimental Methodology	18
20.1 Quantities measured	18
20.2 Definition of a “processor” p	18
20.3 Controls	19
20.4 Strong scaling (fixed problem, more processors)	19
20.5 Weak scaling (more processors, proportionally larger problem)	19
20.6 GPU roofline (single device, problem size grows)	20
20.7 Datasets: PISCOT v1.1 vs v1.2	20
20.8 Hardware configurations	20
20.9 Reporting	20
21 Theoretical Efficiency: Reference Curves for the Scaling Plots	21
21.1 CPU: efficiency model under shared memory bandwidth	21
21.2 GPU: what the theoretical curve is when $p = 1$	22
21.3 Wave granularity: how many fits run at the same instant	23

1 Purpose

This document describes the full algorithm used to estimate daily dewpoint temperature (**TD**) from minimum temperature (**TMIN**) for many spatial locations (**ID**). The design follows a PRAM-style view: independent operations are marked with **pardo**.

The project uses an anomaly local linear regression model:

$$TD_{anom}(t) = TD(t) - TD_{clim}(ID, doy)$$
$$TMIN_{anom}(t) = TMIN(t) - TMIN_{clim}(ID, doy)$$

For each spatial **ID** and each day-of-year **doy**, the model fits a weighted linear regression using **TMIN** anomaly and lagged anomaly variables.

Why anomalies, in plain terms. Raw temperature has a strong, predictable seasonal cycle: every location is warmer in summer and colder in winter. If we modelled raw **TD** against raw **TMIN**, most of the apparent relationship would just be “both follow the seasons” — not the physical link we care about. So we first subtract each location’s *typical* value for that day of year (its **climatology**), leaving the **anomaly**: how far above or below normal today is. The regression then learns the genuine day-to-day coupling between dewpoint and minimum temperature, having removed the season. We fit a *separate* small regression for each (**ID**, **doy**) so the relationship is allowed to change by location and by time of year, and we borrow strength from neighbouring days of the year (a local window of half-width **h**) so each fit has enough samples. The end products are: a climatology table, the fitted regression coefficients per (**ID**, **doy**), and optionally forecast **TD** values.

2 PRAM Model

PRAM (Parallel Random Access Machine) is the standard idealized model for reasoning about parallel algorithms: many processors share one memory, and we count how the work spreads across them. We use the **CREW** (“Concurrent Read, Exclusive Write”) variant, which matches this pipeline exactly:

- **Concurrent read**: many processors may read the *same* shared input tables at once (e.g. every worker can read the climatology). Reads do not conflict.
- **Exclusive write**: no two processors write to the *same* output location. Here that is enforced naturally — each worker owns a disjoint output bucket / **ID** / day-of-year, so there are never write collisions to coordinate.
- **Reductions** (sums, means, counts) are treated as parallel reductions: combining **m** values costs about $\log m$ steps rather than **m**, because the values are added pairwise up a tree.
- **pardo** marks a loop whose iterations are independent and may therefore run in parallel. It is the central annotation in this document: wherever you see **pardo**, there is no dependency between iterations, so a scheduler is free to spread them across all available workers.

The bucket-year design is important because it prevents 300k independent **ID** tasks from repeatedly scanning the same historical parquet files. Instead of each **ID** re-opening every monthly file, the data is grouped once into **B** buckets, and each worker reads its bucket’s compact files a single time — turning a storm of tiny, repeated reads into a few large sequential ones.

3 Symbols

Symbol	Meaning
N	Number of spatial locations, approximately 300,000 IDs
Y	Number of training years, approximately 40
D	Days per year, at most 366

Symbol	Meaning
T	Total days per ID, $T = Y * D$
R	Total rows per climate variable, $R = N * T$
B	Number of ID buckets, for example 1024
p	Number of available processors or worker slots
h	Local DOY half-window, default $h = 11$
W	Samples per local regression, $W = O(Y * h)$
F	Number of regression features plus intercept, constant here
Q	Number of failed DOYs to repair, $Q \leq D$
H	Forecast horizon in days

In this project, D , h , and F are small constants in practice, but they are shown explicitly in the complexity formulas.

4 How to Read the Complexity Formulas

Every algorithm below is described with three kinds of cost. Reading them is easier if you keep their plain-language meaning in mind:

- **Work W (sequential cost).** The *total* number of basic operations the algorithm performs, regardless of how many processors run it. This is what you would pay on a single processor. It never gets smaller by adding hardware — it is the intrinsic size of the computation.
- **Parallel time T_p (wall-clock with p processors).** How long the algorithm takes when p workers run in parallel. The recurring shape is

$$T_p \approx \underbrace{\frac{W}{p_{eff}}}_{\text{work shared out}} + \underbrace{\text{overhead}}_{\text{coordination, I/O, dependencies}}.$$

The first term is the work divided among the workers (perfect speedup). The second term is what stops speedup from being perfect: combining partial results (a *reduction*, costing about $\log$ of the number of items), reading and writing files, scheduler bookkeeping, and any step that must wait for an earlier one.

- **Span T_∞ (the critical path).** The time even with *unlimited* processors — the length of the longest chain of steps that must happen in order. A small span means the algorithm is “wide” (highly parallel); a large span means something is inherently sequential.

Two notations recur and are worth reading literally:

- A $+ \log(\dots)$ term is the cost of a parallel **reduction** — e.g. summing many numbers into one by repeated pairwise addition takes about $\log$ steps. It is small but non-zero, which is why T_p never reaches exactly W/p .
- $\min(p, B)$ appears wherever the work is cut into B independent **bucket tasks**. You can only run as many buckets at once as you have workers *and* as you have buckets, so the usable parallelism is $\min(p, B)$. If $p > B$, the extra workers have no bucket to take and sit idle — the origin of the rule $p_{\{eff\}} \leq B$ discussed in the HPC section.

So, in one sentence: W says how big the computation is, T_p says how much wall-clock time the cluster can buy back, and the overhead terms say why that speedup eventually stops.

5 Algorithm 1: Full TDEW Pipeline

```
1  Algorithm TDEW-PIPELINE
2  Input:
3    TD monthly files for all IDs and dates
4    TMIN monthly files for all IDs and dates
5    Grid table containing the expected IDs
6    Parameters: B buckets, h local DOY window, p processors
7
8  Output:
9    Daily climatology per (ID, doy)
10   Bucket-year merged training dataset
11   Regression coefficients per (ID, doy)
12   Optional repair coefficients for incomplete DOYs
13   Optional TD predictions
14
15  1  climatology <- BUILD-CLIMATOLOGY(TD, TMIN)
16  2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
17  3  bucketed_climatology <- SHARD-CLIMATOLOGY(climatology, B)
18  4  coefficients <- TRAIN-COEFFICIENTS(bucketed_training,
19                                     bucketed_climatology,
20                                     h,
21                                     B)
22  5  incomplete_doy <- CHECK-COMPLETENESS(coefficients, Grid)
23  6  if incomplete_doy is not empty then
24  7    patch <- RERUN-FAILED-DOYS(bucketed_training,
25                                bucketed_climatology,
26                                incomplete_doy,
27                                h,
28                                B)
29  8    coefficients <- PATCH-COEFFICIENTS(coefficients,
30                                         patch,
31                                         incomplete_doy)
32  9    incomplete_doy <- CHECK-COMPLETENESS(coefficients, Grid)
33 10 end if
34 11 if forecasting is requested then
35 12   bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN future, B)
36 13   predictions <- FORECAST-TD(coefficients,
37                                bucketed_climatology,
38                                bucketed_training,
39                                bucketed_future_tmin,
40                                H,
41                                B)
42 14 end if
43 15 return coefficients, predictions
```

Reading the pipeline. The flow is: first learn each location's seasonal normals (**climatology**, line 1); reshape the raw monthly files into compact per-bucket files so later stages read efficiently (**build bucket-year dataset** and **shard climatology**, lines 2–3); fit the local regressions that are the scientific output (**train coefficients**, line 4); then a quality-control loop (lines 5–10) that checks whether every day of year was successfully fitted for every location and, if not, retrains *only* the missing days and patches them in; and

finally an optional **forecast** (lines 11–14) that rolls the fitted model forward over a future horizon. Steps run in sequence, but each step is internally parallel (its `pardo` loops). The later algorithms expand each of these lines.

6 Algorithm 2: Build Climatology

The climatology is the historical mean for each (ID, doy).

```

1  Algorithm BUILD-CLIMATOLOGY
2  Input:
3      TD rows: (ID, date, TD)
4      TMIN rows: (ID, date, TMIN)
5
6  Output:
7      climatology rows: (ID, doy, TD_clim, TMIN_clim)
8
9  1  for each climate variable V in {TD, TMIN} pardo do
10  2      for each monthly file m of V pardo do
11  3          read rows from file m
12  4          for each row r pardo do
13  5              r.doy <- day_of_year(r.date)
14  6              emit partial record (r.ID, r.doy, r.value)
15  7          end for
16  8      end for
17  9      for each key (ID, doy) pardo do
18  10          sum_V(ID, doy) <- parallel sum of values with this key
19  11          count_V(ID, doy) <- parallel count of values with this key
20  12          clim_V(ID, doy) <- sum_V(ID, doy) / count_V(ID, doy)
21  13      end for
22  14 end for
23  15 join TD_clim and TMIN_clim by (ID, doy)
24  16 return climatology

```

Complexity:

$$W_{clim} = O(R)$$

$$T_p = O(R/p + \log R)$$

Space:

$$S_{clim} = O(ND)$$

What the costs mean. The work $O(R)$ says we touch every row of every climate variable exactly once — there is no way to compute averages without looking at all the data, so this is as cheap as it can be. The parallel time $O(R/p + \log R)$ says the row-touching splits perfectly across p workers (R/p), plus a small $\log R$ tail for combining the per-key partial sums into final means (the reduction). The output is one number per (ID, doy) pair, hence memory $O(ND)$ — far smaller than the raw input, because 40 years of daily values collapse into a single seasonal curve per location.

7 Algorithm 3: Build Bucket-Year Training Dataset

This is the main redesign. The expensive monthly TD/TMIN merge is paid once. Rows are then distributed to deterministic ID buckets and compacted by year.

```

1  Algorithm BUILD-BUCKET-YEAR-DATASET
2  Input:
3      TD monthly files
4      TMIN monthly files
5      Number of buckets B
6
7  Output:
8      bucket-year training files:
9      bucket b, year y -> rows (ID, date, TD, TMIN, doy)
10
11  1  for each year y pardo do
12  2      for each month m in year y pardo do
13  3          TD_m <- read TD rows for month m
14  4          TMIN_m <- read TMIN rows for month m
15  5          M_m <- hash join TD_m and TMIN_m by (ID, date)
16  6          for each row r in M_m pardo do
17  7              r.doy <- day_of_year(r.date)
18  8              r.bucket <- r.ID mod B
19  9              emit r to temporary partition (bucket = r.bucket, year = y)
20 10          end for
21 11      end for
22 12      for each bucket b in 0..B-1 pardo do
23 13          read all temporary rows for (bucket b, year y)
24 14          sort rows by (ID, date)
25 15          write compact file for (bucket b, year y)
26 16      end for
27 17 end for
28 18 return bucket-year training dataset

```

Complexity with hash joins:

$$W_{bucket} = O(R)$$

If sorting inside each bucket-year shard is included:

$$W_{bucket-sort} = O\left(R + \sum_{b,y} n_{b,y} \log n_{b,y}\right)$$

With balanced buckets, $n_{\{b,y\}}$ approx $(N/B)D$, so:

$$W_{bucket-sort} = O\left(R + BY \cdot \frac{ND}{B} \log\left(\frac{ND}{B}\right)\right) = O\left(R \log\left(\frac{ND}{B}\right)\right)$$

Ideal parallel time:

$$T_p = O\left(\frac{R}{p} + \log\left(\frac{ND}{B}\right)\right)$$

Peak memory per bucket-year compaction:

$$S_{bucket-year} = O\left(\frac{ND}{B}\right)$$

The idea in one picture. A *bucket* is just a deterministic group of IDs: $\text{bucket} = \text{ID} \bmod B$. We pay the expensive TD $\leftrightarrow$ TMIN merge once, then file every row into its bucket (and year). Afterwards, any stage that needs an ID's data reads one small, already-merged, already-sorted file instead of re-scanning the whole archive.

Reading the formulas. The work $O(R)$ again says we pass over every row once. The $\log(ND/B)$ factor that appears when sorting is included is the cost of ordering the rows *inside* each bucket-year shard; because each shard holds only about ND/B rows (not all R), that log is of a small quantity, so sorting adds very little. The parallel time has the same R/p work-sharing plus that small $\log(ND/B)$ reduction. Crucially, peak memory is $O(ND/B)$, **not** $O(R)$: a worker only ever holds one bucket-year shard at a time, so increasing B *lowers* the memory each worker needs. That is the lever that keeps 300k IDs $\times$ 40 years from ever having to fit in one machine's RAM.

8 Algorithm 4: Shard Climatology by Bucket

```

1  Algorithm SHARD-CLIMATOLOGY
2  Input:
3    climatology rows (ID, doy, TD_clim, TMIN_clim)
4    Number of buckets B
5
6  Output:
7    bucketed climatology files
8
9  1  for each climatology row c pardo do
10  2    c.bucket <- c.ID mod B
11  3    emit c to bucket c.bucket
12  4  end for
13  5  for each bucket b in 0..B-1 pardo do
14  6    write climatology rows for bucket b
15  7  end for
16  8  return bucketed climatology

```

Complexity:

$$W_{\text{clim-shard}} = O(ND)$$

$$T_p = O(ND/p + \log B)$$

Peak memory with streaming:

$$S_{\text{clim-shard}} = O(ND/B)$$

If the full climatology is loaded at once, peak memory becomes $O(ND)$.

Why this step exists. Sharding does no arithmetic — it only *moves* each climatology row to the bucket its ID belongs to, so that the training stage finds an ID's normals in the same bucket as its training rows and reads both with one pass. The cost is therefore a single streaming pass ($O(ND)$ work) plus a $\log B$ term for routing rows to B destinations; streaming keeps only one bucket's worth ($O(ND/B)$) in memory at a time.

9 Algorithm 4b: Shard Future TMIN by Bucket

Forecasting needs the exogenous future TMIN over the prediction horizon laid out by the same buckets as the coefficients, so that each forecast task reads its inputs once. This mirrors the bucket-year preprocessing of Algorithm 3, but for a single variable and only over the forecast horizon.

Let $R_{\text{fut}} = N * H$ be the number of future TMIN rows over the horizon H .

```

1  Algorithm SHARD-FUTURE-TMIN-BY-BUCKET
2  Input:
3      future TMIN monthly files over the forecast horizon
4      Number of buckets B
5
6  Output:
7      bucketed future-TMIN files: bucket b -> rows (ID, date, TMIN)
8
9  1  for each month m in the horizon pardo do
10  2      read future TMIN rows for month m
11  3      for each row r pardo do
12  4          r.bucket <- r.ID mod B
13  5          emit r to temporary partition (bucket = r.bucket)
14  6      end for
15  7  end for
16  8  for each bucket b in 0..B-1 pardo do
17  9      write compact future-TMIN file for bucket b
18  10 end for
19  11 return bucketed future TMIN

```

Complexity:

$$W_{fut-shard} = O(R_{fut}) = O(NH)$$

$$T_p = O(NH/p + \log B)$$

Peak memory with streaming:

$$S_{fut-shard} = O(NH/B)$$

In short. This is the same routing step as Algorithm 4, but applied to the future **TMIN** that drives the forecast, and only over the **H** forecast days rather than the full history. Laying it out by the *same* buckets means each forecast task later reads its coefficients, climatology, history, and future **TMIN** all from one bucket, once.

10 Algorithm 5: Train Anomaly Coefficients

For each **ID** and each target **day**, fit a weighted regression over a circular day-of-year neighborhood.

What one fit actually does. Suppose we want the model for location **i** on day-of-year **d** (say day 60). We do not use only day-60 data — a single day across 40 years is too few points. Instead we gather every observation whose day-of-year lies within **h** days of **d** (the *local window*), wrapping around the year end so that day 365 is a neighbour of day 1 (*circular distance*). Points nearer to day **d** get more influence through a **tricube weight** that smoothly falls to zero at the window edge, so the fit reflects “around day 60” while still being centred on day 60. On that weighted neighbourhood we solve a small weighted least-squares regression of the **TD** anomaly on the **TMIN** anomaly and a few lagged anomalies. The result — the regression coefficients — is stored for (**i**, **d**). We repeat this for every day of year and every location, which is why the total count of fits is enormous even though each one is tiny.

```

1  Algorithm TRAIN-COEFFICIENTS
2  Input:
3      bucket-year training dataset
4      bucketed climatology
5      Local DOY half-window h

```



```

6   Number of buckets B
7
8   Output:
9   coefficients per (ID, doy)
10
11  1  for each bucket b in 0..B-1 pardo do
12  2      train_b <- read all bucket-year training rows for bucket b
13  3      clim_b <- read climatology rows for bucket b
14  4      for each ID i in bucket b pardo do
15  5          X_i <- rows of train_b for ID i
16  6          C_i <- rows of clim_b for ID i
17  7          sort X_i by date
18  8          for each row t in X_i pardo do
19  9              TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
20 10             TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
21 11         end for
22 12         for each row t in X_i pardo do
23 13             create lag features:
24 14                 TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
25 15         end for
26 16         for each target day d in 1..366 pardo do
27 17             N_d <- rows where circular_distance(doy(t), d) <= h
28 18             remove rows in N_d with missing target or missing features
29 19             if size(N_d) >= minimum_sample_count then
30 20                 for each row t in N_d pardo do
31 21                     weight(t) <- tricube(circular_distance(doy(t), d) / h)
32 22                 end for
33 23                 beta <- weighted_least_squares(N_d)
34 24                 write coefficients for (ID i, doy d)
35 25             end if
36 26         end for
37 27     end for
38 28     write coefficient file for bucket b
39 29     write failure log for bucket b, if needed
40 30 end for
41 31 return coefficient dataset

```

Per local regression:

$$W_{one-fit} = O(WF^2 + F^3)$$

Here F is constant, so:

$$W_{one-fit} = O(W) = O(Yh)$$

Where $O(WF^2 + F^3)$ comes from. Solving a weighted least-squares fit means forming the $F \times F$ normal-equations matrix $X^T W X$ from W weighted sample rows (that is the WF^2 term — each of F^2 matrix entries sums over W rows) and then solving the resulting $F \times F$ linear system (the F^3 term, the cost of a small matrix factorization). Because the feature count $F = 5$ is a fixed constant here, both terms collapse to a constant times W , so **one fit costs $O(W) = O(Yh)$** — proportional to how many neighbourhood samples it sees.

All coefficient training:

$$W_{train} = O(NDW) = O(NDYh)$$

That is simply *one fit* ($O(Yh)$) times the *number of fits* (N locations $\times$ D days of year). With $N \approx 3 \times 10^5$ and $D \approx 366$, that is roughly 1.1×10^8 independent fits — this product is the pipeline’s dominant cost and the main target of the scaling experiments.

Ideal PRAM span if buckets, IDs, and DOYs are fully parallel:

$$T_{\infty} = O(\log W + F^3)$$

Reading the span. If we had unlimited processors, all 1.1×10^8 fits could run at once, so the only time left is *one fit’s* critical path: a $\log W$ reduction to sum the neighbourhood into the normal equations, plus the constant F^3 solve. In other words the algorithm is extremely “wide” — almost nothing is inherently sequential inside training. The reason real runs are far slower than T_{∞} is purely that we have p workers, not infinitely many, and that data must be read from disk.

With p processors:

$$T_p = O(NDYh/p + \log(Yh))$$

Practical bucket-parallel execution has at most B outer tasks, so:

$$T_p = O\left(\frac{NDYh}{\min(p, B)} + \text{I/O overhead}\right)$$

Why $\min(p, B)$ and not p — and what this means for a GPU. In the implementation the unit of parallel scheduling is the **bucket**: one bucket is one task handed to one worker. So the number of buckets running *at the same time* can be no larger than the number of workers p and no larger than the number of buckets B — hence $\min(p, B)$. This is **bucket-level** (outer) parallelism, and it is the only thing $\min(p, B)$ bounds.

There is a second, independent level of parallelism *inside* each bucket task: the $\approx (N/B) \cdot D$ individual 5×5 fits in that bucket are mutually independent. How that inner level is exploited is exactly what differs between CPU and GPU:

- **CPU.** Each worker is a single-threaded process (BLAS pinned to one thread), so it works through its bucket’s fits one at a time. The inner level is *not* parallelized per worker; speedup comes entirely from running more buckets at once. Here p = number of worker processes ($\approx$ cores), and you want **many** buckets ($B \geq 4p$) so no core goes idle.
- **GPU (A100).** Each GPU is one worker, so at the bucket level p = number of GPUs (often just 1–8) and $\min(p, B)$ is small. But a GPU does not process a bucket’s fits one at a time: it assembles the $X^T W X$ normal equations for *all* $\approx (N/B) \cdot D \approx 10^5$ fits in the bucket and solves them in a single **batched** call, spreading them across thousands of CUDA cores. That inner batch width — on the order of 10^5 fits offered to the device per batched call, cleared in milliseconds — is where the GPU’s speed comes from, and **$\min(p, B)$ does not bound it**. The practical implication reverses the CPU rule: a GPU wants buckets *large enough* that each batch fills the device, so one does not push B very high for GPU runs.

In both cases the *total* work $O(NDYh)$ is identical; the hardware only changes how the two levels of parallelism are filled. The “Hardware is just more processors” section below makes this precise, and the scaling experiments measure which hardware clears the 1.1×10^8 fits fastest.

Peak memory per bucket task:

$$S_{train-bucket} = O\left(\frac{NT}{B} + \frac{ND}{B}\right)$$

The first term is the bucket training time series. The second term is the bucket climatology.

11 Algorithm 6: Check Completeness

```
1  Algorithm CHECK-COMPLETENESS
2  Input:
3    coefficients
4    expected ID count from grid
5
6  Output:
7    incomplete DOYs
8
9  1  expected_count <- number of IDs in grid
10 2  for each coefficient row c pardo do
11 3    emit key c.doy with value c.ID
12 4  end for
13 5  for each doy d in 1..366 pardo do
14 6    observed_count(d) <- distinct count of IDs emitted for d
15 7    if observed_count(d) < expected_count then
16 8      mark d as incomplete
17 9    end if
18 10 end for
19 11 return incomplete DOYs
```

Let $C = ND$ be the complete number of coefficient rows.

$$W_{check} = O(C)$$

$$T_p = O(C/p + \log C)$$

Space:

$$S_{check} = O(C)$$

What it checks. A (ID, doy) fit can be skipped if its neighbourhood had too few valid samples (line 19 of training). This step is a sanity audit: for each day of year, count how many locations actually got a coefficient and compare to how many the grid says should exist. Any day of year missing some locations is flagged for repair. It is a cheap single pass over the coefficient table ($O(C)$), and its output is tiny — a short list of at most D days to redo.

12 Algorithm 7: Repair Failed DOYs

Only the incomplete DOYs are retrained. If Q is small, this is much cheaper than retraining all 366 DOYs.

```
1  Algorithm RERUN-FAILED-DOYS
2  Input:
3    bucket-year training dataset
4    bucketed climatology
5    incomplete DOYs Q
6    Local DOY half-window h
7
8  Output:
9    patch coefficient dataset
10
11 1  for each bucket b in 0..B-1 pardo do
```

```

12 2    train_b <- read all bucket-year training rows for bucket b
13 3    clim_b <- read climatology rows for bucket b
14 4    for each ID i in bucket b pardo do
15 5        recompute anomalies and lags for ID i
16 6        for each failed day d in Q pardo do
17 7            fit weighted local regression for (ID i, doy d)
18 8            write patch coefficients for (ID i, doy d)
19 9        end for
20 10    end for
21 11 end for
22 12 return patch coefficient dataset

```

Complexity:

$$W_{repair} = O(NQYh)$$

$$T_p = O(NQYh/p + \log(Yh))$$

Why repair is cheap. Repair is structurally identical to training (same buckets, same weighted fits) but restricted to the Q flagged days instead of all D . So its cost is the training cost with D replaced by Q : $O(NQYh)$. When only a handful of days failed ($Q \ll D$), this is a tiny fraction of a full retrain — we redo the few broken days, not all 366.

13 Algorithm 8: Patch Coefficients

```

1  Algorithm PATCH-COEFFICIENTS
2  Input:
3    base coefficient dataset
4    patch coefficient dataset
5    incomplete DOYs Q
6
7  Output:
8    corrected coefficient dataset
9
10 1 for each bucket b in 0..B-1 pardo do
11 2    base_b <- read coefficient rows for bucket b
12 3    patch_b <- read patch rows for bucket b
13 4    base_keep <- rows in base_b whose doy is not in Q
14 5    corrected_b <- union(base_keep, patch_b)
15 6    remove duplicate (ID, doy), keeping the patch row
16 7    sort corrected_b by (ID, doy)
17 8    write corrected coefficients for bucket b
18 9  end for
19 10 return corrected coefficient dataset

```

Complexity:

$$W_{patch} = O(ND + NQ)$$

$$T_p = O((ND + NQ)/p)$$

Peak memory per bucket:

$$S_{patch-bucket} = O(ND/B + NQ/B)$$

What patching does. Rather than editing the big coefficient table in place, each bucket rebuilds itself: keep the rows whose day of year was *not* repaired, add the freshly repaired rows, and drop any duplicate (ID, doy) in favour of the repair. This is pure bookkeeping — a read, a filter, a union, a sort, a write per bucket — so its cost is just proportional to the table size plus the small patch, with no new model fitting.

14 Algorithm 9: Optional Forecasting (bucketed)

Forecasting is parallel across buckets, and across IDs within a bucket, but recursive through time for each ID: day t depends on the previously predicted TD anomaly, so the horizon loop cannot be parallelized within one ID. Like training, the work is organized by bucket so that each coefficient, climatology, history, and future-TMIN shard is read exactly once. The future-TMIN shards are produced by Algorithm 4b; the history (used only to seed the first two lags) is read from the bucket-year training dataset.

```

1  Algorithm FORECAST-TD
2  Input:
3    bucketed coefficients per (ID, doy)
4    bucketed climatology per (ID, doy)
5    bucketed history (to seed lags) and bucketed future TMIN
6    forecast horizon H
7    Number of buckets B
8
9  Output:
10   predicted TD for each (ID, date)
11
12  1  for each bucket b in 0..B-1 pardo do
13    2    coeff_b <- read coefficient rows for bucket b
14    3    clim_b <- read climatology rows for bucket b
15    4    hist_b <- read history rows for bucket b
16    5    ftmin_b <- read future TMIN rows for bucket b
17    6    for each ID i in bucket b pardo do
18      7      initialize lagged TD/TMIN anomalies from hist_b(i)
19      8      for forecast day t from 1 to H do          // sequential in t
20        9        d <- day_of_year(date_t)
21        10       beta <- coeff_b(i, d)
22        11       x <- current TMIN anomaly and lagged anomaly features
23        12       TD_anom_hat <- beta dot x
24        13       TD_hat <- clim_b(i, d) + TD_anom_hat
25        14       update lagged anomalies with TD_anom_hat and TMIN_anom(t)
26        15       write prediction (ID i, date_t, TD_hat)
27        16     end for
28      17   end for
29    18   write prediction file for bucket b
30  19 end for
31  20 return predictions

```

Complexity:

$$W_{forecast} = O(NH)$$

Practical bucket-parallel time (at most B outer tasks, recursive over the horizon within each ID):

$$T_p = O\left(\frac{NH}{\min(p, B)} + H\right)$$

Why forecasting has a + H that training does not. Training fits are mutually independent, so they parallelize completely. Forecasting is different *within* one location: to predict day t we feed in the *predicted* anomaly from day $t-1$ (the lag features). Day 2 needs day 1, day 3 needs day 2, and so on, so the H forecast days for a single ID must be computed *in order* — they cannot be split across workers. That unavoidable chain of length H is the + H term. We still parallelize fully across the millions of *locations* (and across buckets), so the work $O(NH)$ divides by $\min(p, B)$; only the per-ID time-stepping stays sequential. As with training, the bucket structure bounds the effective parallelism by $p_{\text{eff}} \leq B$, and on a GPU the same two-level picture applies — many IDs’ recursions advance one time-step together as a batched operation across CUDA cores.

Peak memory per bucket task:

$$S_{\text{forecast-bucket}} = O\left(\frac{NH}{B} + \frac{ND}{B}\right)$$

15 Old Design vs Bucket-Year Design

The original Colab-style design trained one task per ID. Each task discovered, opened, and filtered the same monthly parquet files.

Let $S = 12Y$ be the number of monthly source files per variable.

Old I/O task overhead:

$$O(NS)$$

For $N = 300,000$ and $Y = 40$, this means roughly:

$$300,000 \times 480 = 144,000,000$$

file-touch attempts per variable family before considering actual row reads.

The bucket-year design changes the I/O pattern:

- Source files are scanned once during preprocessing.
- Training reads compact bucket-year files.
- Workers write disjoint bucket outputs.
- No large driver-side concatenation is required.

New source scan overhead:

$$O(S)$$

New model training work:

$$O(NDYh)$$

So the redesign does not remove the statistical work of fitting many local regressions, but it removes the repeated source-file scanning that made the previous implementation slow and fragile.

16 Summary Complexity Table

The table collects the three costs for every phase. Read each row as: “*this is how big the phase is (w), how its wall-clock time shrinks with p workers (T_p), and how much memory one worker needs.*” The phases with $\min(p, B)$ in their T_p (train, repair, forecast) are the bucket-parallel compute stages whose speedup saturates once p reaches B ; the others ($/p$ only) are I/O-bound passes that scale with workers until the filesystem becomes the limit. The single dominant entry is **train coefficients**, $O(NDYh)$.

Phase	Sequential work	Ideal parallel time with p processors	Peak memory target
Build climatology	$O(R)$	$O(R/p + \log R)$	$O(ND)$ output
Build bucket-year dataset	$O(R)$ or $O(R \log(ND/B))$ with sorting	$O(R/p + \log(ND/B))$	$O(ND/B)$ per compaction
Shard climatology	$O(ND)$	$O(ND/p + \log B)$	$O(ND/B)$ streaming
Shard future TMIN	$O(NH)$	$O(NH/p + \log B)$	$O(NH/B)$ streaming
Train coefficients	$O(NDYh)$	$O(NDYh/\min(p, B) + \log(Yh))$	$O(NT/B + ND/B)$ per bucket
Check completeness	$O(ND)$	$O(ND/p + \log(ND))$	$O(ND)$ or bucket streaming
Repair Q failed DOYs	$O(NQYh)$	$O(NQYh/\min(p, B) + \log(Yh))$	same as training bucket
Patch coefficients	$O(ND + NQ)$	$O((ND + NQ)/p)$	$O(ND/B + NQ/B)$ per bucket
Forecast horizon H	$O(NH)$	$O(NH/\min(p, B) + H)$	$O(NH/B)$ per output bucket

17 Overall Complexity

The full pipeline is a sequence of parallel phases, so the total work is the sum of the work of each phase. The dominant terms are:

- bucket-year preprocessing,
- full coefficient training,
- optional failed-DOY repair,
- optional forecasting.

Using $R = NYD$, the total sequential work for preprocessing plus full training without repair and without forecasting is:

$$W_{total} = O\left(R \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Equivalently:

$$W_{total} = O\left(NYD \log\left(\frac{ND}{B}\right) + NDYh\right)$$

If sorting cost inside bucket-year files is ignored or treated as an implementation detail, the simpler expression is:

$$W_{total} = O(NYD + NDYh) = O(NDYh)$$

because h is the main local-regression multiplier.

If repair is needed for Q failed DOYs, add:

$$W_{repair-total} = O(NQYh)$$

So full training plus repair is:

$$W_{total+repair} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q)\right)$$

If forecasting is also executed for horizon H , add:

$$W_{forecast-total} = O(NH)$$

Therefore the most complete end-to-end expression is:

$$W_{end-to-end} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q) + NH\right)$$

For parallel execution, the phases are sequentially dependent, but each phase uses `pardo` internally. With p processors and B bucket tasks, the practical parallel time is:

$$T_p = O\left(\frac{NYD \log(ND/B)}{p} + \frac{NDYh}{\min(p, B)} + \frac{NQYh}{\min(p, B)} + \frac{NH}{\min(p, B)} + H + \log(NYD)\right)$$

Term by term, that parallel time reads as: build the bucket-year dataset ($NYD \log(ND/B) / p$), do the full training ($NDYh / \min(p, B)$, the big one), repair the failed days if any ($NQYh / \min(p, B)$), run the forecast ($NH / \min(p, B)$), pay the unavoidable per-ID forecast recursion ($+ H$), and pay one global reduction ($+ \log(NYD)$). The preprocessing terms divide by p (they are limited by the filesystem, not buckets), while the three compute stages divide by $\min(p, B)$ (they are limited by the bucket count).

The $+ H$ term remains because forecasting is recursive over time for each ID. If forecasting is not executed, remove $NH/\min(p, B) + H$. If no failed DOYs exist, remove the $NQYh / \min(p, B)$ term.

For this project, the practical bottleneck after the bucket-year redesign is:

$$O(NDYh)$$

That is the cost of fitting local regressions for many IDs and all days of year. The redesign mainly reduces repeated I/O from the old Colab-style approach.

18 HPC Interpretation

The total work complexity does not depend on p because it measures the total number of operations required by the algorithm. The same joins, anomaly calculations, local regressions, repairs, and forecasts must be computed whether the job runs on one processor or on a university high performance computing cluster.

On an HPC system, the relevant measure for execution time is the parallel runtime:

$$T_p = O\left(\frac{W}{p_{eff}} + \text{I/O overhead} + \text{scheduling overhead} + \text{sequential dependencies}\right)$$

where p_{eff} is the effective number of processors that the algorithm can use. It can be smaller than the physical number of available cores because of bucket count, memory bandwidth, file-system bandwidth, scheduler overhead, and dependencies between pipeline stages.

For the current bucket-based training design:

$$p_{eff} \leq B$$

This is because the main training stage creates one large parallel task per bucket. If $B = 1024$, the training stage can naturally expose up to about 1024 parallel bucket tasks. If the HPC provides fewer than 1024 worker slots, the cluster can be used efficiently. If the HPC provides far more than 1024 worker slots, the extra processors will not reduce training time much unless we increase the number of buckets or add another level of parallelism inside each bucket, for example by splitting work by `ID` or by groups of `doyle`.

A practical rule for an HPC deployment is:

$$B \geq 4p$$

where p is the number of worker slots planned for the run. This gives the scheduler enough bucket tasks to balance slow and fast workers. However, B should not be made arbitrarily large because too many small buckets create many small files and increase scheduling overhead.

Example starting points:

Planned worker slots	Suggested bucket count
64	512 or 1024
128	1024 or 2048
256	2048 or 4096
512	4096 or 8192

Therefore, for a university HPC, the project should be evaluated using both:

- total work, which describes the size of the scientific computation; and
- parallel runtime, which describes how much wall-clock time can be reduced by using the cluster.

The expected speedup is strong while p_{eff} increases, but it eventually saturates because of I/O bandwidth, scheduler overhead, bucket count, and the recursive forecast dependency over the horizon H .

19 Practical Notes for This Machine

With about 300k IDs, 40 years, and 32 logical CPU threads, the best strategy is to keep the bucket count much larger than the number of workers. For example, $B = 1024$ gives roughly $N/B \approx 293$ IDs per bucket, which keeps each bucket task large enough to amortize overhead but small enough to fit in memory.

19.1 Hardware is just more processors

The complexity above is hardware-agnostic: the total work $W = O(NDYh)$ is the same whether the pipeline runs on CPU cores or on a GPU. Hardware changes only two things: the number of parallel processors p it exposes, and the constant factor c of a single local fit. The parallel runtime is always

$$T_p = c \cdot \frac{W}{p_{\text{eff}}} + \text{overhead}.$$

The per-(ID, doy) weighted least squares is embarrassingly parallel: there are about $N * D$ (roughly 1.1×10^8) independent fits, each a small $F = 5$ feature weighted regression over $W = O(Yh)$ neighborhood rows. This maps equally well to:

- **CPU:** many single-threaded bucket worker processes (one per core), where the bucket structure bounds $p_{\text{eff}} \leq B$; or
- **GPU:** a device that evaluates thousands of these tiny fits concurrently once they are batched, by assembling the $F \times F$ weighted normal equations $(X^{\text{top } W} X)^{\beta} = X^{\text{top } W} y$ for many (ID, doy) pairs and solving them in a single batched call.

Two levels of parallelism — and which one a GPU fills. It is worth being precise about what “more processors” means here, because the two hardwares add parallelism at different levels:

- *Outer (bucket) level* — how many buckets run at once. This is what $\min(p, B)$ bounds. On CPU, p is the number of worker processes ($\approx$ cores); on GPU, p is the number of **GPUs** (one `dask-cuda` worker per device), often just 1–8.

- *Inner (fit) level* — how many of a bucket’s $\approx (N/B) \cdot D \approx 10^5$ independent fits are evaluated together. On CPU this is 1 (each worker fits one at a time, BLAS pinned). On GPU this is the **batch width**: the device factors 10^5 stacked 5×5 systems at once. $\min(p, B)$ does **not** bound this level.

Concretely, on the benchmark device (a MIG a100_3g.20gb slice, 42 SMs) the compiled solve kernel needs 127 registers per thread, so the register budget — not the nominal 2048-threads-per-SM limit — is what binds: 512 threads fit per SM, and the device holds $42 \times 512 = 21,504$ fits in flight at once. A whole bucket’s $\approx 10^5$ fits are offered to the device in one batched call and cleared in about 5 hardware **waves** of 21,504 fits each, within milliseconds (the “Wave granularity” subsection of the Theoretical Efficiency section derives and validates these numbers from the benchmark results). So one GPU gives $p = 1$ at the bucket level but $\sim 2 \times 10^4$ fit-lanes per wave — the speed lives at the inner level. This also reverses the bucket-count guidance: CPU wants **many** buckets ($B \geq 4p$) to keep cores busy, whereas GPU wants buckets **large enough** that each batch fills the device, so B is not pushed high for GPU runs.

So a GPU is, in PRAM terms, simply a source of a larger *effective* p (with a different constant c): a modest outer p multiplied by a very large inner batch. Which of CPU or GPU yields the smaller $c \cdot T_p$ for this workload is an empirical question, answered by the scaling experiments below, not by the complexity model.

The remaining, hardware-independent, speedups still come from:

- avoiding repeated parquet scans (the bucket-year redesign),
- using bucket-year files,
- keeping worker memory bounded,
- parallelizing by bucket,
- and repairing only failed DOYs instead of repeating all training.

20 Experimental Methodology

This section defines how the implementation is evaluated on the university HPC. The goal is to measure, as a function of the number of processors p , three standard parallel-performance quantities, and to do so on both CPU and GPU hardware and on both dataset versions.

20.1 Quantities measured

For a fixed problem size, let $T(p)$ be the wall-clock time of a phase (or of the end-to-end pipeline) using p processors. We report:

$$\text{Execution time: } T(p) \quad \text{Speedup: } S(p) = \frac{T(1)}{T(p)} \quad \text{Efficiency: } E(p) = \frac{S(p)}{p}$$

In plain terms: **speedup** answers “how many times faster did p workers make it?” and **efficiency** answers “what fraction of those workers’ time was actually useful?” Ideal speedup is $S(p) = p$ (8 workers $\rightarrow 8\times$ faster) and ideal efficiency is $E(p) = 1$ (100% of every worker busy). Real efficiency drops below 1 because some work is sequential or workers wait on each other or on the disk. $S(p)$ is ideally linear and $E(p)$ is ideally 1. The PRAM analysis predicts the departure from ideal: because the training and forecast stages are bucket-parallel with $p_{\text{eff}} \leq B$, efficiency stays high while $p \leq B$ and then degrades; I/O bandwidth, scheduler overhead, and the sequential pipeline dependencies set the practical ceiling.

20.2 Definition of a “processor” p

For the CPU study, p counts independent worker slots:

Hardware	One processor (p unit)	How p is varied
CPU (SLURM)	one single-threaded Dask worker process (one core, BLAS pinned to 1 thread)	<code>dask_jobqueue.SLURMCluster</code> scaled to p workers

The **GPU** study is different. The target deployment (KHIPU) exposes exactly **one** A100 **MIG slice** (a100_3g.20gb; account limit `gres/gpu=1`), so there is **no GPU p-sweep** — `p_{GPU}=1` always. Reporting $S(p)/E(p)$ over GPUs is therefore not meaningful here. Instead the GPU is characterised by **throughput and a roofline** as the *problem size* grows: the x-axis is **M = number of fits** ($\approx N \cdot 366$) and curves are drawn per **N = IDs per bucket** (the data volume). This measures the inner (batched) parallelism that is the GPU’s actual source of speed, and is compared head-to-head with the CPU sweep in wall-clock terms.

20.3 Controls

To make the measurements reproducible and fair:

- **Baseline T(1)** is a single worker with `threads_per_worker = 1` and BLAS thread pools pinned to 1 (`OMP_NUM_THREADS=1`, etc.), as in `run_pipeline.sh`.
- **Bucket count** is held at $B \geq 4, p_{\max}$ for the whole sweep so the bucket granularity is never the limiting factor before `p` reaches its maximum.
- **Repeated trials**: each (p, n) point is run k times (e.g. $k = 3$) and the median wall-clock is reported, to suppress filesystem and scheduler jitter.
- **Storage**: worker scratch / spill goes to node-local fast storage (`local_directory = $TMPDIR`); input parquet lives on the shared filesystem.
- **Warm vs cold cache** is recorded; cold-cache (first-touch) runs are reported separately because the I/O-bound preprocessing phases are cache sensitive.
- Only the phase under test is timed (with `time.perf_counter` around the driver call); inputs from earlier phases are precomputed once and reused.

The two scaling studies below ask two different questions. **Strong scaling** fixes the problem and adds workers — “can the cluster make *this* job finish sooner?” **Weak scaling** grows the problem in step with the workers — “if we double both the data and the machines, does the time stay flat?” Strong scaling exposes the $p_{\text{eff}} \leq B$ ceiling; weak scaling isolates the overhead that grows with the number of workers.

20.4 Strong scaling (fixed problem, more processors)

Hold the problem size fixed at the full grid (or a fixed representative subset of N IDs) and vary `p` (CPU only; the GPU has a single device — see the GPU roofline below):

$$p \in \{1, 2, 4, 8, 16, 32\} \quad (\text{CPU; KHIPU account limit } \textit{cpu} = 32)$$

For each `p` we record $T(p)$ for the dominant **train-coefficients** phase (and, separately, the I/O-bound build-bucket and climatology phases, and optionally forecast). From these we compute $S(p)$ and $E(p)$ and plot all three against `p`. The expected curve rises near-linearly while $p \leq B$ and the workload is compute-bound, then flattens — directly testing the $p_{\text{eff}} \leq B$ prediction.

20.5 Weak scaling (more processors, proportionally larger problem)

Hold the work per processor constant by scaling the number of trained IDs with `p`:

$$n(p) = n_0 \cdot p$$

where n is the number of IDs trained. The ID subset is selected deterministically through the existing `bucket_ids` parameter (each bucket holds $\approx N/B$ IDs, so choosing $\lceil 4p \rceil$ buckets gives a controlled $n(p)$). Weak-scaling efficiency is

$$E_{\text{weak}}(p) = \frac{T(1, n_0)}{T(p, n_0 p)}$$

which is ideally 1. Because the per-(ID, doy) fits are independent, the compute part of training is expected to weak-scale well; departures isolate I/O and scheduling overhead that grow with the number of workers.

20.6 GPU roofline (single device, problem size grows)

Because only one GPU (MIG `a100_3g.20gb`) is available, the GPU is evaluated as a **throughput / roofline** study rather than a processor sweep. The GPU training of a bucket has three stages, benchmarked separately (`benchmark_gpu_pipeline.py`, timed with CUDA events):

1. **assemble** — scatter raw rows into per-`(ID, doy)` 5×5 sufficient statistics ($X^T W X$, $X^T W y$, $\Sigma w y^2$);
2. **convolve** — $2h+1$ weighted circular rolls over the day-of-year axis (the local window);
3. **solve** — the fused one-thread-per-fit 5×5 Cholesky kernel.

We sweep $N = \text{IDs per bucket}$ (so $M = \text{\#fits} \approx N \cdot 366$ on the x-axis) and report, per stage: execution time vs M , throughput (fits/s) and **time-per-fit** vs M , achieved **GFLOPS**, and **arithmetic intensity** $I = \text{FLOPs} / \text{bytes}$ placed on a **roofline**. The attainable performance is $\min(\pi, I \cdot \beta)$ for compute peak π (FP64) and memory bandwidth β (HBM, measured empirically on the slice with a large copy).

A short analytic FLOP/byte model (FP64) is used. The solve kernel does ≈ 180 FLOP/fit and moves ≈ 296 B/fit (read `A[5x5]`, `b[5]`, `\Sigma wy^2`; write `\beta[5]`, `r^2`), so its intensity is $I \approx 0.6$ FLOP/byte; the assemble (scatter) and convolve (streaming rolls) stages have even lower intensity ($I \approx 0.1$). All three lie far to the **left of the roofline ridge** ($\pi/\beta \approx 5$ FLOP/byte), so the GPU pipeline is **memory-bandwidth bound** — its speed comes from the inner batch width ($\approx 10^5$ fits cleared per wave), not FP64 throughput. The single-GPU end-to-end time is also compared directly against the CPU sweep (wall-clock, same workload).

20.7 Datasets: PISCOT v1.1 vs v1.2

The full strong- and weak-scaling protocol is run on both TMIN dataset versions, **PISCOT v1.1** (folder `tmin_v11`, figshare 16372509, 1981–2016) and **PISCOT v1.2** (folder `tmin_v12`, figshare 20533715, 1981–2020); the shared dewpoint target is PISCOT v1.1 TDEW (`td`). The comparison window is the overlap **1981–2016**. The two versions are supplied to the same pipeline through the input path / variable-folder parameters (`--base`, `--td-var`, `--tmin-var`) writing to separate result roots, so no code differs between runs. We compare:

- **Performance:** $T(p)$, $S(p)$, $E(p)$ curves for v1.1 vs v1.2 (the work W differs only if N , Y , or data density differ between versions).
- **Science:** distributions of the fitted coefficients and of the prediction skill — RMSE, MAE, bias, Pearson r , and **cosine similarity** (predicted-vs-observed TD , and v1.1-vs-v1.2 agreement) — to assess how the dataset version affects the estimated model, independent of runtime.

20.8 Hardware configurations

Configuration	Cluster	Axis swept
CPU multi-node	<code>dask_jobqueue.SLURMCluster</code> (one process per core, <code>B \geq 4p_{\text{max}}</code>)	p worker processes $\in \{1, \dots, 32\}$
GPU single device	one MIG <code>a100_3g.20gb</code> slice, in-process (<code>client=None</code> , no <code>dask-cuda</code>)	$M = \text{\#fits}$ (problem size), per $N = \text{IDs/bucket}$

20.9 Reporting

For each dataset we produce, on **CPU**: (1) a table of $T(p)$, $S(p)$, $E(p)$ for the strong-scaling sweep; (2) a table of $E_{\text{weak}}(p)$; and (3) three plots — execution time vs p , speedup vs p (with the ideal $S(p)=p$ reference), and efficiency vs p . On **GPU** (single device): per-stage execution-time, throughput, time-per-fit and GFLOPS vs M ; a **roofline** (achieved GFLOPS vs arithmetic intensity with the compute/bandwidth ceilings); and a **CPU-vs-GPU** wall-clock comparison. Together these quantify how the implementation scales and which hardware minimizes wall-clock time for this workload.

21 Theoretical Efficiency: Reference Curves for the Scaling Plots

The empirical plots above show *measured* efficiency. This section derives the *theoretical* efficiency curves that are overlaid (dashed) on those plots, in the same spirit as the asymptotic $\mathcal{O}$ -notation reduction for the parallel K-means exercise: start from a time model, form $E(p) = T_1 / (p T_p)$, and reduce with the problem size held constant. The two hardware targets need two different derivations, because they expose parallelism on two different axes: the CPU sweeps **processors** p at fixed problem size, while the single GPU sweeps **problem size** N at fixed $p = 1$.

21.1 CPU: efficiency model under shared memory bandwidth

Ruling out the fixed-overhead model first. The PRAM prediction for the training stage is $T_p = \mathcal{O}(NDYh/p + \log(Yh))$ (with $B \geq 4p$, so $\min(p, B) = p$). If the departure from ideal efficiency were caused by that fixed additive overhead (call it σ), the efficiency would be

$$E(N, p) = \frac{W t_c}{W t_c + p \sigma}, \quad W = NDYh,$$

which *improves* as N grows: a larger problem dilutes a fixed overhead. The measurements show the opposite — the $E(p)$ curves for $N = 512, 1024$, and 2048 IDs are superimposed almost exactly. So the binding constraint is not a fixed overhead; it must be a resource that is *consumed in proportion to the work itself and shared by all p workers*. On one node that resource is the **memory bus**.

Time model. Each unit of work (one row touched during a local fit) needs compute time t_c and moves q bytes. Compute divides among the p cores; the Wq bytes of memory traffic must all cross the one shared bus of bandwidth β no matter how many cores are active:

$$T_p = \underbrace{\frac{W t_c}{p}}_{\text{compute, shared out}} + \underbrace{\frac{W q}{\beta}}_{\text{bus, not shared}}.$$

Speedup and efficiency. With $T_1 = W(t_c + q/\beta)$ and the single dimensionless ratio

$$\gamma = \frac{q/\beta}{t_c} \quad (\text{memory time per unit of work / compute time per unit of work}),$$

the speedup and efficiency are

$$S(p) = \frac{T_1}{T_p} = \frac{p(1 + \gamma)}{1 + \gamma p}, \quad E(p) = \frac{S(p)}{p} = \frac{1 + \gamma}{1 + \gamma p}.$$

Asymptotic reduction. Note something *stronger* than the “ N constant” premise of the K-means exercise: the work $W = NDYh$ **cancels entirely** — $E(p)$ depends only on γ and p , not on the problem size. That is exactly the empirical signature in the strong-scaling plot (three problem sizes, one curve). In $\mathcal{O}$ -notation:

$$E(p) = \mathcal{O}\left(\frac{1}{1 + \gamma p}\right) = \mathcal{O}\left(\frac{1}{p}\right) \quad (p \gg 1/\gamma),$$

and the speedup saturates at

$$S(\infty) = \lim_{p \rightarrow \infty} \frac{p(1 + \gamma)}{1 + \gamma p} = 1 + \frac{1}{\gamma}.$$

The asymptotic class is the same $\mathcal{O}(1/p)$ as the broadcast K-means version, but the physical cause differs: there it was communication volume growing with p ; here it is p cores contending for one memory bus.

Fitting γ . The model has a single parameter, estimated by least squares over the pooled measured points of *all three* problem sizes (the model says one γ must fit them all — a falsifiable claim, not a

convenience). On the KHIPU Job A data this gives $\gamma \approx 0.063$, i.e. memory stalls cost about 6% of the compute time per unit of work, and

p	2	4	8	16	32
E(p) pre-dicted	0.944	0.849	0.707	0.529	0.352
E(p) mea-sured (N=512/1024/2048)	0.91–0.94	0.82–0.83	0.70–0.71	0.56	0.34–0.35

with a saturation ceiling $S(\infty) \approx 17$ — consistent with the measured speedup still creeping upward at $S(32) \approx 11$. This is the red dashed curve on the efficiency plot (drawn by `analyze_scaling.py: fit_gamma / theory_efficiency`), the analogue of the $0(1/p)$ reference line in the K-means exercise.

21.2 GPU: what the theoretical curve is when $p = 1$

The GPU plot (`gpu_pipeline_time_vs_M.png`) has **no processor axis**: there is one device, and the x-axis is the problem size $M = N \cdot 366$ fits. An “ideal efficiency $E(p) = 1$ ” line is therefore meaningless there — with $p_{\text{GPU}} = 1$, $E(p)$ is identically 1 by construction and says nothing. The outer $\min(p, B)$ level of the PRAM model is pinned at 1; all the performance lives in the *inner* batch level, and the correct theoretical reference for that level comes from the roofline.

Bandwidth floor. The roofline analysis showed every stage sits far below the ridge ($I \approx 0.08\text{--}0.6$ FLOP/byte vs ridge ≈ 6.6), so each stage is memory-bandwidth bound and its execution time is bounded below by the time needed just to move its bytes:

$$T_{\text{bound}}(M) = \frac{Q_s(M)}{\beta},$$

where $Q_s(M)$ is the analytic bytes-moved model of stage s (the `assemble_cost`, `convolve_cost`, `solve_cost` functions in `benchmark_gpu_pipeline.py`) and β is the *measured* HBM bandwidth of the slice (633 GB/s). Since $Q_s(M) \propto M$, the floor is $T_{\text{bound}} = 0(M/\beta)$: a straight dashed line of slope 1 in the log-log plot, one per stage. This — not an efficiency line — is the theoretical curve drawn on `gpu_pipeline_time_vs_M.png`.

The single-GPU analogue of efficiency. The quantity that plays the role of $E(p)$ is the **bandwidth utilization**

$$\eta(M) = \frac{T_{\text{bound}}(M)}{T_{\text{meas}}(M)} = \frac{\text{achieved GB/s}}{\beta} \in (0, 1],$$

the vertical gap between each measured curve and its dashed floor. A simple latency-plus-bandwidth time model explains its shape: with a fixed per-launch cost t_0 ,

$$T(M) \approx t_0 + \frac{Q(M)}{\beta} \quad \Rightarrow \quad \eta(M) = \frac{Q(M)/\beta}{t_0 + Q(M)/\beta} \xrightarrow{M \rightarrow \infty} 1,$$

so η is low while the batch is too small to hide the launch cost and rises toward a plateau once M fills the device. The measurements match: `convolve` is flat at ≈ 7 ms from $N = 64$ to $N = 293$ (pure launch overhead, $\eta \approx 0.1$) and reaches $\eta \approx 0.5$ at $N = 1024$; `assemble` holds $\eta \approx 0.6$ throughout; the `solve` kernel reaches $\eta \approx 0.9$ at the largest M — essentially at its roofline. In 0-notation the story is one line: on a single GPU,

$$T(M) = \left(\alpha_0 + \frac{M}{\beta} \right), \quad \eta(M) = \left(\frac{1}{1 + t_0 \beta / M} \right) \rightarrow 1,$$

the mirror image of the CPU case — there, efficiency *decays* as $0(1/p)$ because workers contend for one bus; here, utilization *saturates* toward the bandwidth ceiling as the batch grows, which is precisely why the GPU wants **large buckets** (the **B** guidance of the HPC section) while the CPU wants many.

21.3 Wave granularity: how many fits run at the same instant

The batched solve does not execute all M fits simultaneously; the hardware runs them in **waves**, and the benchmark results contain everything needed to count them:

1. The solve kernel uses **one thread per fit**, and the compiled kernel needs **127 registers per thread** — a *register* is a thread’s private fast storage (here holding the 5×5 system and the Cholesky intermediates). This number is chosen by the CUDA compiler, reported by `cupy`, and logged as the `num_regs` column of `gpu_kernel.csv`.
2. GPU threads are scheduled in **warps** of 32 threads, so one warp needs $32 \times 127 \approx 4,096$ registers, which the hardware rounds up to $4,096$.
3. Each SM owns a pool of $65,536$ registers, so $65,536 / 4,096 = 16$ warps = 512 threads fit per SM — register-limited to 25% of the nominal 2048-thread-per-SM capacity.
4. The MIG slice has **42 SMs**, so the device holds $42 \times 512 = 21,504$ fits in flight at any instant, and

$$\text{waves}(M) = \left\lceil \frac{M}{21,504} \right\rceil.$$

M (fits)	waves	solve time (ms)	μs per wave
23,424	2	0.0266	13.3
107,238	5	0.0727	14.5
187,392	9	0.1065	11.8
374,784	18	0.1915	10.6
800,000	38	0.3835	10.1

The last column is the model check: measured solve time divided by the wave count settles to a constant $\approx 10 \mu\text{s}$ per wave as M grows (the higher values at small M are the launch overhead t_0 of the previous subsection), and 21,504 fits every $10 \mu\text{s}$ reproduces the measured saturated throughput of $\approx 2 \times 10^9$ fits/s. At full scale, the 300k-ID domain ($M_{\text{total}} \approx 1.1 \times 10^8$ fits) therefore ran about **5,150 solve waves** in total, ≈ 5 per $B = 1024$ bucket.

No dedicated “waves vs M ” plot is needed: $\text{waves}(M)$ is the straight line $M/21,504$ by construction, so it carries no information beyond this table. The visual evidence of wave execution is already in the time-vs- M plot — above one wave the solve curve grows linearly in M , at a fixed $\approx 10 \mu\text{s}$ per step of 21,504 fits. (The wave count applies to the `solve` RawKernel only; `assemble` and `convolve` are `cupy` library operations whose launch shapes were not recorded, and being bandwidth-bound their floor is set by $Q(M)/\beta$ regardless of wave structure.)